



斯坦福 CS336 从零开始的语言建模

讲座 18：推理系统与循环 Transformer

客座讲座 – Dan Fu

课程笔记整理

2026 年春季

目录

1 引言：从训练到推理的视角转换	4
1.1 讲座定位与动机	4
1.2 模型能力的革命性跃迁	4
1.3 GPU：新时代的石油	5
1.4 本章小结	5
2 Token 的生命周期：推理系统概览	6
2.1 推理系统的核心组件	6
2.2 工作负载的多样性	6
2.3 时间约束与服务目标	7
2.4 本章小结	7
3 预填充与解码：两个截然不同的阶段	7
3.1 预填充：计算密集型前向传播	7
3.2 解码：内存带宽密集型自回归生成	8
3.3 连续批处理：提升吞吐量的核心机制	9
3.4 本章小结	9
4 大规模部署的挑战与优化	10
4.1 KV 缓存：推理的命脉	10
4.1.1 前缀共享与树形结构	10
4.1.2 KV 缓存的层次管理	10
4.2 模型并行策略	11
4.3 预填充-解码分离	12
4.4 缓存感知预填充-解码解耦（CPD）	12
4.5 生产环境的罕见错误	13
4.5.1 重复输出循环	14
4.5.2 工具调用失败	14
4.5.3 随机中文输出	14
4.6 本章小结	15
5 巨型内核：解码推理的极致优化	15
5.1 解码的内核级挑战	15
5.2 聚内核：从独立操作到全局融合	16
5.3 ThunderKittens：细粒度 GPU 控制框架	17
5.4 本章小结	18

6 PROSE: 循环 Transformer 与新的扩展维度	18
6.1 问题提出: 扩展的唯一方式?	18
6.2 循环 Transformer 的基本概念	19
6.3 训练稳定性的挑战	19
6.3.1 数学分析: 从非线性到线性近似	20
6.3.2 重新参数化实现稳定训练	20
6.4 三维扩展法则	21
6.5 本章小结	22
7 总结与延伸	23
7.1 讲座核心回顾	23
7.2 研究方向的启示	23
7.3 全栈创新的可能性	24
7.4 Q&A 精华	24
7.4.1 预训练模型能否直接循环使用?	24
7.4.2 循环模型对推理内存的影响?	24
7.4.3 不同应用场景的最佳架构差异	24
7.5 结语	25

1 引言：从训练到推理的视角转换

1.1 讲座定位与动机

本课程的前 17 讲主要聚焦于如何训练语言模型，使其具备强大的生成与理解能力。而在本次客座讲座中，Dan Fu 将视角转向一个同样关键却常被低估的问题：一旦拥有了这些模型，如何高效地将其应用于推理（inference）？

Dan Fu 目前隶属于 UCSD（加州大学圣地亚哥分校）的研究实验室，同时也是 Together AI 的研究成员。Together AI 是一家提供 GPU 推理、微调等服务的 AI 云平台，拥有深厚的科研背景（包括 Percy Liang 等学者）。因此，本次讲座的内容既有学术前沿的研究洞察，也包含大规模生产系统的实战经验。

Dan Fu 将推理系统比喻为“将电力转化为智能的引擎”。正如石油必须通过引擎才能转化为动能，训练好的模型参数也必须通过推理引擎才能转化为用户可用的智能输出。理解推理系统，不仅是部署大模型的工程需求，更是推动算法全栈创新的关键切入点。

核心观点

如果你理解推理本身、推理引擎的设计，以及支撑核心技术的 GPU 内核编程，你就能在机器学习算法的全栈层面实现创新——无论是设计新的路由算法、调度策略、高效内核，还是全新的模型架构。

1.2 模型能力的革命性跃迁

过去数年间，我们在模型及其能力方面经历了一场真正的工业革命：

- **文本与代码生成**：从 GPT-2 级别的文本生成到接近人类水平的代码编写能力；
- **多模态理解**：模型不仅能处理文本，还能生成和理解图像、视频；
- **科学应用**：在 DNA 建模、生物信息学、健康医疗等领域展现出前所未有的潜力。

推动这些能力飞跃的核心因素是**规模扩增（Scaling）**。Dan Fu 以自身经历说明了这一变化的惊人速度：2018 年他攻读博士学位时，最大的模型参数量约为 1 亿；而到了今天，开源模型已达到万亿参数级别，技术前沿甚至可能达到 5 至 10 万亿参数。

历史类比：马粪问题

1902 年的曼哈顿拥有 13 万匹工作马，马粪问题严重到需要召开专门的学术会议来讨论处理方案。然而仅仅 10 年后（1912 年），汽车数量就超过了马匹。这种技术在十年内的彻底转变，与今天语言模型在我们工作和生活中的渗透有着惊人的相似性。Dan Fu 认为，去年（2025 年）可能就是那个转折点——他已经大规模使

用语言模型编写代码，并督促所有学生也这样做。

1.3 GPU：新时代的石油

规模扩增的背后是海量计算资源的支持。GPU 已成为新时代的“石油”，全球有数百亿美元的投资涌向 GPU 基础设施。一些国家的主权财富基金更是将 GPU 资源视为国家稳定的重要组成部分。

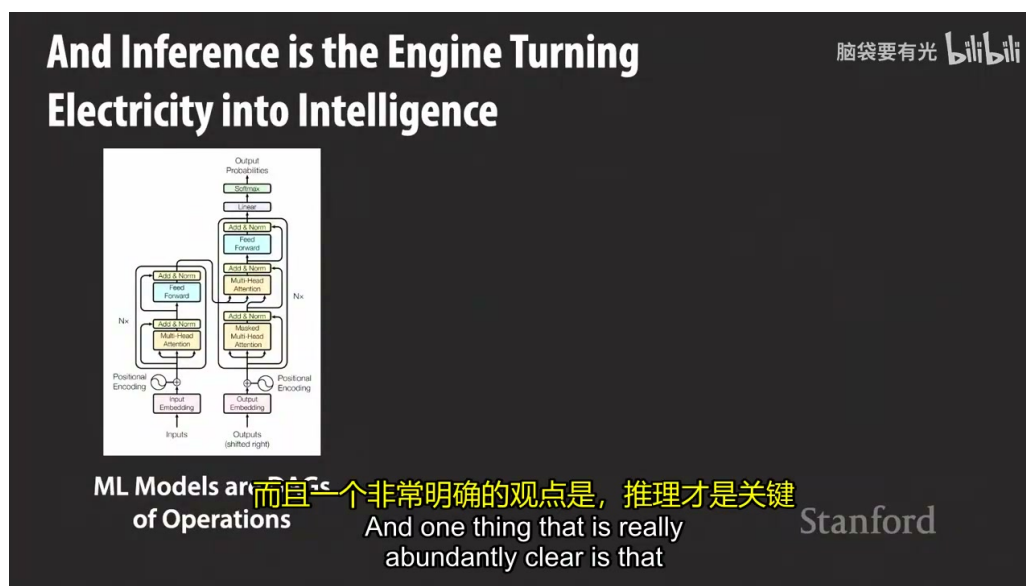


图 1: 推理是将电力转化为智能的引擎。ML 模型本质上是一系列操作的流程图，而推理引擎和 GPU 内核负责将这些数学抽象转化为实际可用的计算。

然而，GPU 本身只是原材料。真正让这些 GPU 焕发光彩的，是推理引擎和 GPU 内核——它们负责将模型参数和计算图高效地映射到硬件上执行。这正是本次讲座要深入探讨的核心主题。

1.4 本章小结

- 训练使模型获得能力，推理使能力转化为可用的智能；
- 规模扩增是能力飞跃的驱动力，但推理效率决定了这些能力能否经济地服务于用户；
- 理解推理系统需要跨层次的视野：从请求调度、模型并行，到 GPU 内核级别的优化。

2 Token 的生命周期：推理系统概览

2.1 推理系统的核心组件

当你向一个 LLM 推理系统发送请求时，这个请求会经历一系列精心设计的阶段。Dan Fu 将其概括为“Token 的生命周期”（Lifetime of a Token）：

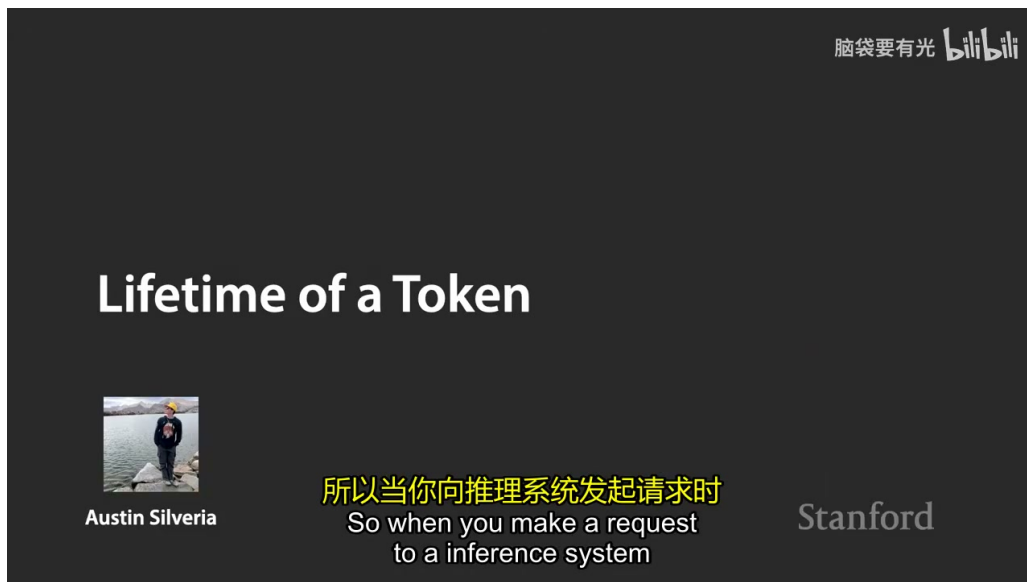


图 2: Token 的生命周期：从用户请求到生成输出的端到端流程。

1. 请求接收与调度：请求首先到达调度器，被分配到不同的 GPU 或机器上；
2. KV 缓存查找：系统检查是否之前见过类似的请求或前缀，以复用已计算的激活值；
3. 核心 ML 计算：执行预填充（prefill）和解码（decode）阶段的核心计算；
4. 并行策略：根据模型规模，在节点间、GPU 间进行张量并行或流水线并行；
5. 输出生成：生成新的 token，进行后处理（安全检查、停止条件判断等），返回给用户。

2.2 工作负载的多样性

生产环境中的推理请求并非均匀同质。Dan Fu 强调了理解工作负载特征的重要性，因为不同的应用场景会产生截然不同的请求模式：

表 1: 不同应用场景下的工作负载特征对比

应用场景	输入长度	输出长度	交互模式
编程助手（Cursor 类）	极长（数万 token）	中等	多轮迭代
文档摘要	超长（整本书）	短	单次
聊天问答	短至中等	中等	多轮
代理工作流	中等	长	异步、工具调用

不要假设统一的工作负载

在设计和优化推理系统时，一个常见的误区是假设所有请求都类似。实际上，编码工作负载与叙事性总结、快速聊天与长时间代理任务之间的差异巨大。忽略这些差异会导致严重的资源错配和性能瓶颈。

2.3 时间约束与服务目标

不同应用对响应时间的要求也截然不同：

- 交互式聊天：需要秒级甚至亚秒级响应，用户期望看到“正在思考”的即时反馈；
- 语音助手：对延迟极其敏感，任何明显的停顿都会破坏用户体验；
- 代理工作流：可以容忍分钟级延迟，因为代理被期望独立完成复杂任务；
- 批处理任务：如文档翻译，延迟要求最低，但吞吐量至关重要。

这些约束共同定义了推理系统的服务等级协议（SLA），并直接影响调度策略和硬件配置的选择。

2.4 本章小结

- 推理系统是一个端到端的流水线，涉及调度、缓存、计算、并行和后处理；
- 工作负载的多样性决定了没有“一刀切”的优化方案；
- 理解应用层面的时间约束是设计推理架构的前提。

3 预填充与解码：两个截然不同的阶段

3.1 预填充：计算密集型前向传播

预填充（Prefill）是指模型首次处理用户输入提示（prompt）的阶段。假设用户发送了 1 万个从未见过的 token，模型需要计算这些 token 的激活值和注意力关系，最终输出

第一个新 token。

从计算特征来看，预填充具有以下几个特点：

- **输入大、输出小**：输入可能长达数万 token，但只输出一个 token；
- **计算密集型**：需要执行完整的前向传播，矩阵乘法密集；
- **类似训练**：计算模式与训练阶段非常相似，只是跳过了反向传播；
- **可高度并行**：由于同时处理大量 token，可以充分利用 GPU 的并行计算能力。

预填充与 Flash Attention

在预填充阶段，Flash Attention 等高效注意力算法可以发挥最大作用。因为此时序列长度较大，内存访问模式相对规则，计算密集的特性使得优化带来的收益非常明显。

3.2 解码：内存带宽密集型自回归生成

解码（Decode）是指模型逐个生成新 token 的阶段。在处理完提示后，模型开始自回归地生成输出：每次生成一个 token，将其追加到序列中，然后重新输入模型以生成下一个 token。

解码阶段的计算特征与预填充截然不同：

- **计算量小**：每次只处理一个（或少数几个）token，矩阵乘法规模很小；
- **内存带宽受限**：每次生成都需要加载完整的模型权重，计算速度受限于内存带宽而非算力；
- **难以并行**：自回归特性使得 token 之间存在严格的顺序依赖；
- **需要 KV 缓存**：为了避免重复计算，必须缓存之前 token 的键值（Key-Value）对。

推测解码

为了缓解解码的串行瓶颈，业界发展了**推测解码（Speculative Decoding）**技术：使用一个小型草稿模型快速生成多个候选 token，然后用大模型并行验证这些候选。如果验证通过，一次可以 accept 多个 token，从而将“一次一步”变为“一次多步”。在理想情况下，推测解码可以将生成速度提升 2-3 倍。

预填充 vs 解码的本质差异

- **预填充**：计算密集型（compute-bound），可充分利用 GPU 算力
- **解码**：内存带宽密集型（memory-bandwidth-bound），速度取决于权重加载速度

这一根本差异意味着：对预填充最优的硬件配置和软件优化策略，对解码往往并非最优。

3.3 连续批处理：提升吞吐量的核心机制

为了在同一 GPU 上高效处理多个请求，现代推理系统采用**连续批处理（Continuous Batching）**技术。与传统深度学习中的静态批处理不同，连续批处理允许在推理过程中动态地加入和移除请求。

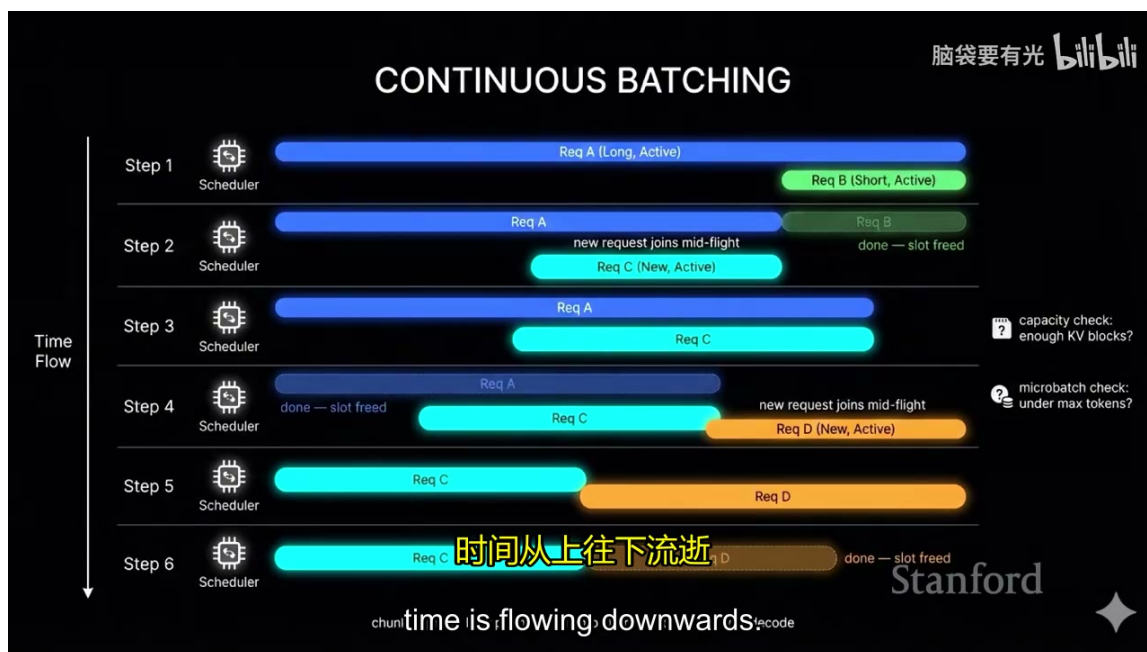


图 3: 连续批处理示意图。时间从上往下流逝，不同颜色的条代表不同请求。系统在每个步骤中同时处理多个请求，并在请求完成后动态释放资源、接纳新请求。

如图3所示，连续批处理的关键特征包括：

1. **动态入队**：新请求可以在任何时候加入当前批次，无需等待前一批次完成；
2. **动态完成**：当一个请求生成结束标记（如 EOS）时，其占用的资源被立即释放；
3. **资源约束检查**：系统持续监控 KV 缓存容量和最大 token 限制；
4. **多样化请求共存**：长请求和短请求可以在同一个时间步内并行处理。

3.4 本章小结

- 预填充和解码在计算特性上存在根本性差异：前者计算密集，后者带宽密集；
- 连续批处理是提升推理吞吐量的关键技术，允许请求动态进出；

- 理解这两种阶段的差异是后续所有优化策略的基础。

4 大规模部署的挑战与优化

4.1 KV 缓存：推理的命脉

KV 缓存 (Key-Value Cache) 是推理系统中最重要优化机制之一。其核心思想很简单：在多轮对话中，如果用户的前缀输入没有变化，就不需要重新计算其键值激活。

当用户说 “Hi ChatGPT” 时，系统计算并缓存这些 token 的 K 和 V 向量。当用户继续提问时，只需计算新增 token 的 K、V，并与缓存中的历史 K、V 拼接，大大减少了重复计算。

KV 缓存的经济价值

KV 缓存的节省效果极其显著。在真实生产环境中，大量用户以类似方式开始对话（如 “Hi” “Hello”），或者在同一会话中进行多轮交流。有效的 KV 缓存机制可以将这些场景的推理成本降低数倍。

4.1.1 前缀共享与树形结构

为了高效管理 KV 缓存，系统通常采用基于**前缀共享**的数据结构（如基数树/Radix Tree）。当新请求到达时，系统快速检查其前缀是否与已缓存的序列匹配：

- 如果完全匹配，直接复用缓存的激活值；
- 如果部分匹配，复用公共前缀的计算结果，只计算新增部分；
- 如果没有匹配，执行完整的预填充计算。

4.1.2 KV 缓存的层次管理

随着模型上下文长度增长到百万级别，KV 缓存的存储和管理成为核心挑战。生产系统通常采用三级存储策略：

1. **GPU 显存**：最高速的存储，存放当前活跃请求的 KV 缓存；
2. **CPU 内存**：当 GPU 显存不足时，将较旧的 KV 缓存卸载到主内存；
3. **SSD/磁盘**：对于超长时间间隔的对话（如用户两周后返回），将 KV 缓存持久化到 SSD。

操作系统类比

这个三级存储架构（GPU 显存 → CPU 内存 → SSD）与 70-80 年代操作系统的内存管理图表几乎完全相同——当时的问题是如果打开太多应用程序会耗尽 RAM，不得不将程序交换到磁盘上。Dan Fu 打趣道：“你们应该去学一门操作系统课。”事实上，经典的最近最少使用（LRU）淘汰策略在这里仍然相当有效。有操作系统论文证明，LRU 的性能与最优策略相差不超过一倍。当然，最完美的策略是“预测未来”：如果系统知道某个用户即将查看一个月前的聊天记录，就可以提前将该对话的 KV 缓存预加载到 GPU 上。

NVL72 与 CPU 瓶颈

NVIDIA 的新一代 Blackwell 架构推出了 NVL72 系统，将 72 个 GPU 通过超高速互联连接在一起。有趣的是，Jensen Huang（黄仁勋）最近非常关注 CPU 性能——原因之一正是 CPU 性能不足可能成为瓶颈。当百万美元级的 GPU 集群被低价的 CPU 拖慢时，这显然不是理想的投资回报。

随着这种超大规模系统的部署，一个严峻的问题浮出水面：**容错性**。NVL72 中的连接器是塑料而非金属制成的，如果电缆插得太紧会受压变形，导致 NVLink 连接不稳定。这些设备还内置了风扇，设计显得有些奇特。更根本的问题是：如果将一个万亿参数模型分片到 64 个 GPU 上，为数百万用户提供服务、每天处理万亿级别的 token，**当单个 GPU 故障时怎么办？**能否设计出让系统在部分硬件故障时仍然保持服务的机制？这是大规模推理部署中一个极其重要但尚未被充分解决的问题。

4.2 模型并行策略

当模型规模达到万亿参数级别时，单个 GPU 无法容纳整个模型。此时需要采用各种**模型并行策略**：

表 2: 常见模型并行策略对比

并行策略	做法	适用场景
张量并行（TP）	将每层切分到多个 GPU	单节点内高速互联
流水线并行（PP）	将不同层分配到不同 GPU	跨节点扩展
专家并行（EP）	MoE 模型中将专家分散到不同 GPU	稀疏激活的大规模 MoE

并行策略的权衡

选择并行策略时需要在多个维度之间权衡：通信带宽需求、内存占用、负载均衡、容错性等。没有 **universally optimal** 的方案，最佳策略取决于具体的模型架构、硬

件配置和工作负载特征。

4.3 预填充-解码分离

由于预填充和解码的计算特性截然不同，一个基本的优化策略是将它们分离到不同的 GPU 集群上执行：

- **预填充集群：**配置高算力 GPU，最大化 FLOPS 利用率；
- **解码集群：**配置高内存带宽 GPU，最小化权重加载延迟。

这一分离策略已经影响了硬件产业的发展方向。Groq 推出专用推理芯片（LPU）、Cerebras 推出晶圆级引擎、SambaNova 等公司也在布局，NVIDIA 甚至计划在下一代系统中将预填充和解码分配到不同类型的芯片上。这些都反映了业界对“专用硬件处理特定工作负载”这一趋势的认同。

4.4 缓存感知预填充-解码解耦（CPD）

Dan Fu 介绍了一项简单而有效的优化工作——缓存感知预填充-解码解耦（**Cache-aware Prefill-Decode Disaggregation, CPD**）。

其核心洞察是：在真实工作负载中，大约只有 10% 的请求是全新对话（冷启动），而 90% 的请求是已有对话的延续（热启动）。这一比例来源于一个简单的启发式：假设对话平均持续 10 轮，那么每 10 个请求中大约只有 1 个是全新的。两类请求对预填充计算的需求差异巨大：

- **冷请求：**需要处理数千甚至数万 token 的完整预填充，计算开销极大；
- **热请求：**只需处理少量新增 token，大部分计算可从 KV 缓存复用。

Cache-aware prefill-decode disaggregation (CPD)

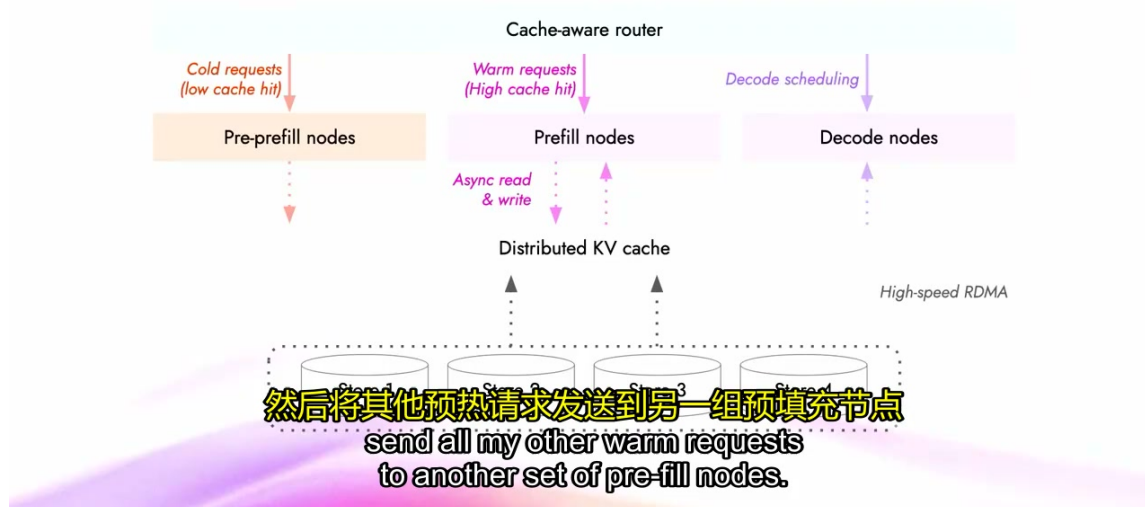


图 4: 缓存感知预填充-解码解耦 (CPD) 架构。冷请求 (低缓存命中率) 被路由到预预填充节点, 热请求 (高缓存命中率) 被路由到预填充节点, 解码则在专门的解码节点上执行。

如图4所示, CPD 通过一个简单的缓存感知路由器 (仅需两行代码), 将不同类型的请求发送到最适合处理它们的节点组。其路由逻辑直观而有效: 一个用户粘贴整本书问“跟我聊聊这本书”的冷请求, 不应该与正在进行简短对话 (如“为什么 $1+1=2$ ”) 的热请求争夺同一组 GPU 资源。实验结果表明, 这一简单优化可以带来高达 **40%** 的服务速度提升。

4.5 生产环境的罕见错误

当推理系统每天处理数万亿 token 时, 即使概率极低的错误也会变得频繁发生。Dan Fu 分享了几类在 **2024 年底**前后、使用某些开源推理引擎时, 在大规模部署中遇到的真实错误:

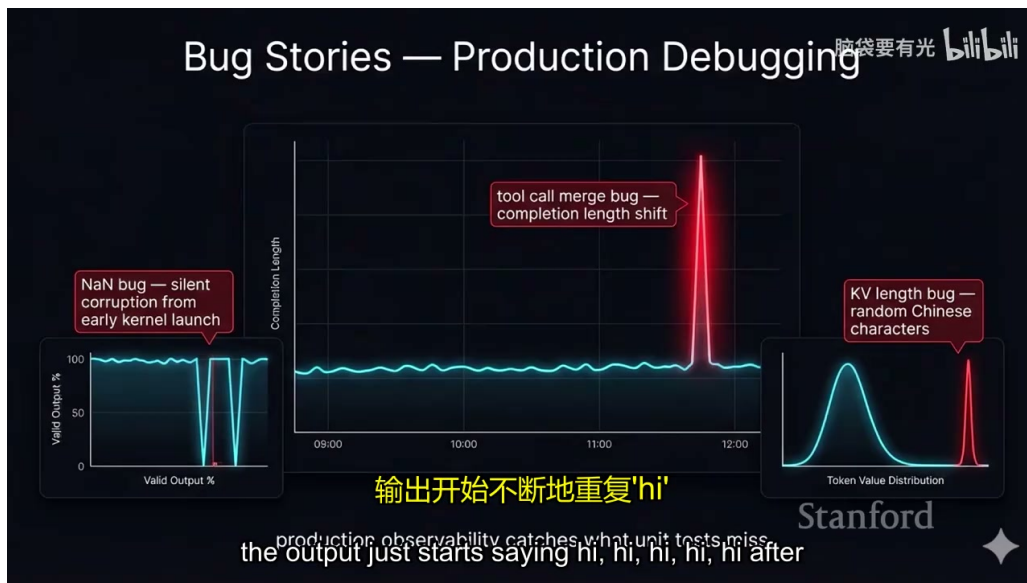


图 5: 大规模推理系统中可能遇到的各类错误和异常行为。

4.5.1 重复输出循环

某些情况下，模型会陷入重复输出同一个词的循环（如不断输出“Hi”或感叹号）。这通常是由于 GPU 内核中的微小 bug 在特定罕见条件下被触发，导致计算中间结果混乱。

4.5.2 工具调用失败

当模型调用外部工具（如网络搜索）时，如果推理引擎没有正确处理工具调用的返回格式，模型可能陷入“请求搜索 → 没有正确返回 → 再次请求搜索”的无限循环，输出长度异常膨胀。

4.5.3 随机中文输出

最有趣的案例是模型突然开始用中文回答英文问题。这一 bug 最初被误认为是量化问题，而且同时影响了数个推理服务提供商。表面上看像是模型被微调成了中文，但根本原因实际上是 GPU 内核中的内存偏移错误：有时内核会从显存读取一些额外的未初始化内存，经过注意力机制处理后，在整个过程结束后恰好产生了一个随机的中文字符。模型随后“以为”用户在用中文提问——“我为什么突然开始用中文思考？用户肯定是在问我中文问题”——于是开始用中文胡言乱语。这个案例的讽刺之处在于：人们以为模型真的被训练成了“中文思维”，而实际上这只是某人代码里的一个偏移错误。

小规模测试的局限性

这些错误的一个共同特点是：它们在小型测试环境中几乎不可能被发现。只有在每天数万亿 token 的生产规模下，这些概率极低的边界情况才会浮现。这提醒我们，推理系统的可靠性评估必须在接近生产规模的负载下进行。

4.6 本章小结

- KV 缓存是推理系统的核心优化，前缀共享和层次化管理是其关键技术；
- 预填充-解码分离是基本的架构优化，CPD 在此基础上通过缓存感知路由进一步提升效率；
- 大规模部署会暴露小规模测试无法发现的罕见错误，需要系统性的监控和诊断机制。

5 巨型内核：解码推理的极致优化

5.1 解码的内核级挑战

如第 3 节所述，解码阶段的主要瓶颈是**内存带宽**而非计算能力。每次生成一个 token 都需要加载整个模型的权重，这使得 GPU 的大量计算单元处于空闲等待状态。

然而，即使认识到带宽瓶颈，传统的推理优化仍然面临一个深层问题：**内核启动开销和空闲时间**。

Challenge: Stragglers and Kernel Launch

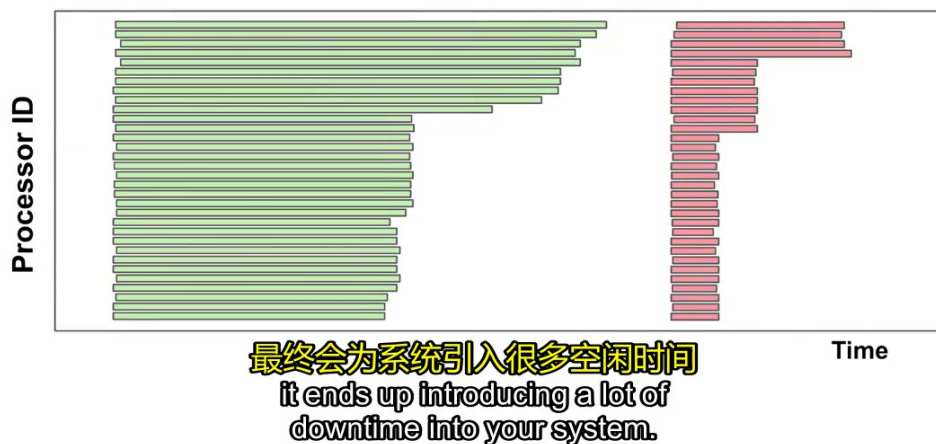


图 6: GPU 处理器时间线展示了传统推理中的空闲时间。Y 轴为 GPU 流式多处理器 (SM), X 轴为时间。绿色条表示有用工作量, 空白区域为空闲等待时间。

图6展示了一个注意力推理内核在 GPU 上的执行情况。可以看到:

- 大量处理器在等待其他操作完成 (空白区域);
- 红色和黄色区域表示内核启动和销毁的开销;
- **尾部效应 (Tail Effect)**: 当一批输入中既有短序列又有长序列时, 短序列的处理器必须等待长序列完成;
- 多个内核之间的间隔逐渐累积, 严重降低了整体效率。

5.2 聚内核: 从独立操作到全局融合

为了应对上述挑战, Dan Fu 团队提出了**聚内核 (MegaKernel)**策略。其核心思想是: 不要将模型中的每个操作视为独立的内核来编写, 而是写一个**单一的大内核**来同时覆盖多个甚至整个层的操作。

聚内核的核心思想

将 GPU 视为一个庞大的**分布式系统**: 有许多任务需要处理, 部分任务之间存在依赖关系。关键在于如何调度这些任务、分配工作, 以最大化 GPU 利用率。

Whole-Model Megakernels

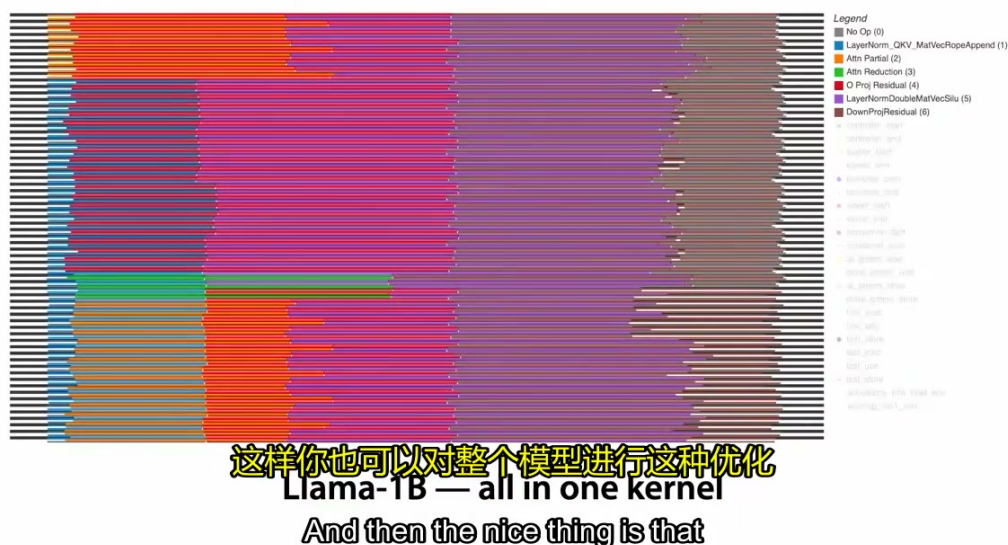


图 7: Llama-1B 模型的整层聚内核执行时间线。不同颜色代表不同操作（LayerNorm、QKV 投影、注意力、O 投影等），展示了操作之间的高度重叠和流水线化。

图7展示了将整个 Transformer 层融合到一个内核中的执行效果。可以观察到：

- 各种操作以高度不规则的方式重叠执行；
- 系统在 QKV 投影和 RoPE 编码仍在运行时，就开始提前加载 KV 缓存到注意力模块；
- O 投影在注意力操作结束前就开始加载权重，实现了精细的流水线化。

注意力优化的巨大潜力

如果仅针对注意力推理部分进行聚内核优化，速度可以提升 30% 到 70%。而将这种融合策略扩展到整个模型层（如图7所示的 Llama-1B 整层融合），则能进一步消除层与层之间的内核启动开销，实现更极致的性能。

5.3 ThunderKittens: 细粒度 GPU 控制框架

为了实现这种级别的内核融合和调度，Dan Fu 团队开发了 **ThunderKittens** 库。它可以被视为比 Triton 更低级的内核编写框架：

- 提供对 GPU 硬件更精细的控制能力；
- 基于指令级抽象，允许每个子内核独立实现；
- 配备大型虚拟化共享内存系统来协调操作执行；

- 作为回报，能够获得接近理论极限的解码推理速度。

实验结果表明，在 H100 GPU 上，ThunderKittens 实现的聚内核达到了 **72%** 的内存带宽利用率——这在 GPU 推理中已接近理论上的极致速度。

为什么需要更低级的控制？

Triton 等高级内核编写工具已经大大简化了 GPU 编程，但它们为了通用性而牺牲了一部分性能。要达到极致的推理速度，需要深入到指令级别来控制：何时加载数据、何时启动计算、如何重叠内存访问和算术运算。ThunderKittens 正是为了这种极致优化而设计的。

5.4 本章小结

- 解码阶段的性能瓶颈不仅是内存带宽，还包括内核启动开销和处理器空闲时间；
- 聚内核通过将多个操作融合为单一内核，实现了操作级的流水线化和重叠执行；
- ThunderKittens 等框架提供了达到理论极限速度所需的细粒度硬件控制。

6 PROSE：循环 Transformer 与新的扩展维度

6.1 问题提出：扩展的唯一方式？

讲座的前半部分聚焦于如何在固定模型架构下优化推理效率。后半部分则将视角转向一个更根本的问题：增加模型参数量和训练数据真的是唯一的扩展方式吗？

当前的主流范式（Chinchilla 扩展法则）告诉我们：要获得更好的模型质量，应该同时增加参数数量 N 和训练数据量 D 。Dan Fu 团队提出的问题是：是否存在第三个可以扩展的维度？

“Claude Mythos”事件与循环模型的突然走红

在 PROSE 发布前大约一周，一位 OpenAI 员工在推特上声称 Claude 是“循环语言模型”，引发了广泛猜测和关注。尽管 Dan Fu 认为这“完全是不正确的”、“他只是在编造”，但这一事件迫使该员工后来发表博客文章承认错误。有趣的是，这个 viral 的误传恰好发生在一个学术团队（马里兰大学 Tom Goldstein 组）发表论文证明循环模型在某些任务上可能优于 Transformer 之后。这个巧合使得“循环 Transformer”突然成为了社区热议的话题，也为 PROSE 的研究提供了额外的动机和关注度。

6.2 循环 Transformer 的基本概念

PROSE（以及更广泛的循环 Transformer 方向）给出的答案是：**循环深度**——即让模型中的某些层重复执行多次。

传统 Transformer 中，token 逐层通过模型，每层的参数只使用一次。而循环 Transformer 的做法是：

1. 选取 Transformer 中的某些块（blocks）作为“循环块”；
2. 在循环块内部，让 token 多次通过同一组参数；
3. 通过控制循环次数，在不增加参数的情况下增加计算量（FLOPs）。



图 8: PROSE 模型的核心问题：除了扩展参数和数据，我们是否可以通过循环深度来提升模型质量？

6.3 训练稳定性的挑战

循环 Transformer 面临的最大挑战是训练不稳定性。Dan Fu 指出：

循环模型的训练陷阱

循环 Transformer 对超参数极其敏感。仅仅是稍微调整学习率，十次中有九次会导致模型无法收敛——损失值飙升、激活范数爆炸到 10^{19} 量级。这种不稳定性是阻碍循环模型被广泛采用的主要障碍。

6.3.1 数学分析：从非线性到线性近似

为了理解不稳定性的根源，Dan Fu 团队对循环块进行了数学分析。他们将循环块分解为三个部分：

- **R**: 一个大的非线性组件（包含注意力、门控、前馈网络等所有复杂操作）；
- **A**: 一个线性变换矩阵，描述每次循环中残差的变换方式；
- **B**: 另一个线性变换矩阵，作用于初始输入向量。

通过这种分解，循环块的动态可以近似表示为：

$$\mathbf{h}_{t+1} = \mathbf{A}\mathbf{h}_t + \mathbf{B}\mathbf{x}_t + \text{nonlinear terms} \quad (1)$$

忽略非线性部分（将其视为扰动），得到一个简单的线性动态系统——这可以用高中微积分求解！闭式解表明，系统的稳定性主要由矩阵 **A** 的谱半径（Spectral Radius）决定：

$$\rho(\mathbf{A}) = \max_i |\lambda_i(\mathbf{A})| \quad (2)$$

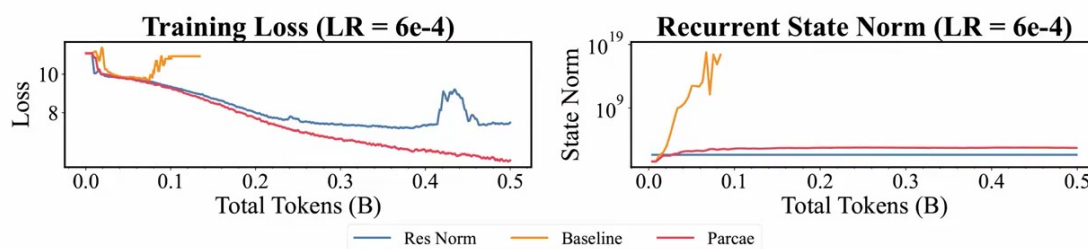
其中 $\lambda_i(\mathbf{A})$ 是 **A** 的特征值。如果 $\rho(\mathbf{A}) > 1$ ，则随着循环次数 t 增加， $\mathbf{A}^t$ 会指数级增长，导致激活值爆炸。

6.3.2 重新参数化实现稳定训练

基于上述分析，PROSE 提出了稳定化策略：

1. 将 **A** 矩阵限制为反对角矩阵（anti-diagonal），确保其幂次趋于零而非爆炸；
2. 对 **B** 矩阵应用简单的线性范数约束；
3. 保证谱半径 $\rho(\mathbf{A}) < 1$ ，从而形成稳定系统。

Stable System, Stable Loss Curves



我们看到损失曲线变得稳定了
what we saw is that you
actually saw a stable loss curve.

图 9: 稳定化前后的训练损失和循环状态范数对比。左图: 无约束基线 (橙色) 在学习率 6×10^{-4} 下损失爆炸; 简单施加范数惩罚的模型 (蓝色) 虽然控制了激活增长, 但产生了损失尖峰 (两种对抗压力的冲突); PROSE 重新参数化模型 (红色) 则保持稳定收敛。右图: 基线的状态范数飙升至 10^{19} , 而 PROSE 模型保持在合理范围。

图9清晰地展示了稳定化的效果。通过重新参数化, 即使使用对普通循环模型来说“致命”的学习率, PROSE 仍能稳定训练。

6.4 三维扩展法则

在解决了训练稳定性问题后, Dan Fu 团队进一步探索了循环次数在扩展法则中的角色。传统扩展法则只考虑两个变量:

$$\hat{\mathcal{L}}(N, D) = E + X \cdot N^{-x} + Y \cdot D^{-y} \quad (3)$$

其中 N 为参数量, D 为数据量 (token 数)。PROSE 将循环次数 R 引入, 形成三维扩展法则:

$$\hat{\mathcal{L}}(N, D, R) = E + X \cdot N^{-x} + Y \cdot D^{-y} + Z \cdot R^{-z} \quad (4)$$

如何阅读缩放曲线

在缩放法则图表中, 曲线的走向揭示了最优扩展策略:

- 向右下方延伸: 应同时增加数据和参数 (当前主流结论)
- 垂直向下: 只需增加数据, 无需更多参数

- 水平向右：只需增加参数，无需更多数据

例如，用 35 万亿 token 训练万亿参数模型的结论正是基于“向右下方延伸”的观察。PROSE 的发现是：在固定参数量时，增加数据的同时也应增加循环次数——即三维同时扩展。

实验发现：

- 在固定参数量的条件下，增加数据量的同时应当增加循环次数；
- 循环次数的缩放遵循与传统扩展类似的幂律关系；
- 最优策略是同时缩放参数、数据和循环深度三个维度。

How to Scale Training FLOPs?

张俊豪 Bilibili

$$\hat{\mathcal{L}}(N, D) = E + X \cdot N^{-x} + Y \cdot D^{-y}$$

What to Scale:

Traditional Scaling Laws: params N, tokens D

- Model size (parameters)
- Data

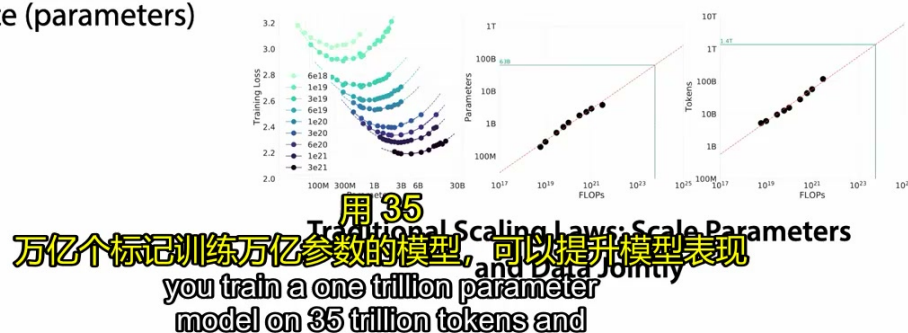


图 10: 传统扩展法则（参数 N 和数据 D ）以及 PROSE 提出的三维扩展概念。图表表明，当增加数据量时，也应当相应增加循环次数以获得最佳性能。

循环扩展的启示

当前主流的大型预训练模型（如 GPT-4、Claude 等）都没有采用循环机制。这意味着它们在固定参数量的条件下，可能未能充分利用可用的计算预算。PROSE 的实验表明，在相同 FLOPs 下，循环模型可以达到比传统 Transformer 更低的验证损失。

6.5 本章小结

- 循环 Transformer 提供了除参数和数据之外的第三个扩展维度——循环深度；

- 训练不稳定性是循环模型的核心障碍，通过数学分析和重新参数化可以有效解决；
- 三维扩展法则（参数、数据、循环）预示着未来模型设计的新方向。

7 总结与延伸

7.1 讲座核心回顾

Dan Fu 的讲座从推理系统的工程实践出发，逐步深入到内核优化和模型架构创新，展现了一条从“让模型跑起来”到“重新思考模型本身”的完整思维路径。

表 3: 讲座核心内容回顾

层次	主题	核心洞察
系统层	Token 生命周期	推理是端到端的系统工程
调度层	连续批处理/CPD	工作负载感知的路由带来 40% 提升
硬件层	预填充-解码分离	不同计算特性需要不同硬件
内核层	聚内核	融合操作消除空闲，接近理论极限
架构层	PROSE 循环模型	循环是第三个可扩展维度

7.2 研究方向的启示

Dan Fu 在讲座中提出了多个值得进一步探索的研究方向：

- 预训练模型的循环增强：**一个令人惊讶的初步发现是，将已有预训练模型（如 Qwen）的某些层进行简单循环后，在数学推理等任务上获得了性能提升。Dan Fu 对此感到“有些不安”——“我不明白为什么会这样”——但这暗示现有模型可能“欠优化”了其参数的计算利用率。团队正在深入研究这一现象的机理。
- 硬件-架构协同设计：**随着专用推理芯片（如 Groq、Cerebras）的兴起，模型架构设计需要更紧密地考虑目标硬件特性。例如，NVIDIA 的 NVFP4 格式仅限 NVIDIA 芯片使用，而 AMD 芯片则使用 MXFP4 格式；中国的模型开发者也在考虑华为芯片的特性。内存容量、量化格式、通信带宽等因素都应该成为架构选择的输入。
- 自动化内核生成：**当前，一名全职且有才华的内核工程师一年只能为 2-3 个模型编写巨型内核，且仅覆盖批次大小 1-16 的范围——“如果增大到批次大小 17，你就得重新来过”。Together AI 等机构正在开发自动化编译器来降低这一门槛，但这仍然是一个极具挑战性的开放问题。

4. **多 GPU 聚内核**：目前的聚内核主要优化单 GPU 上的执行。但事实证明，NCCL 通信调用也可以被融合进聚内核中。DeepSeek 团队已经发布了面向 MoE 推理层的超大内核，融合了部分通信操作。随着模型规模增长，未来的趋势可能是“计算的一小部分成为小型内核，但整个模型不一定都有这样的内核”。

7.3 全栈创新的可能性

讲座的核心信息是：**理解推理系统可以开启机器学习全栈创新的机会**。这种创新可以发生在任何层次：

- **算法层**：新的路由算法处理更多流量（如 CPD）；
- **调度层**：以不同方式调度流量，优化延迟-吞吐量权衡；
- **内核层**：新的内核实现大幅提高系统局部运行速度；
- **架构层**：新的模型结构（如循环 Transformer）减少通信量、降低部署成本。

7.4 Q&A 精华

7.4.1 预训练模型能否直接循环使用？

可以，而且效果出人意料。有研究者发现，将 Qwen 模型的某些层简单循环后，在数学任务上获得了性能提升。这说明循环机制可能不仅仅是训练阶段的扩展手段，也可能是提升已有模型能力的后处理工具。Dan Fu 团队正在深入研究这一现象的机理。

7.4.2 循环模型对推理内存的影响？

循环结构的推理优势在于：

- 参数量减少意味着可容纳更多 KV 缓存；
- 更小的模型可以用更少的 GPU 部署，减少通信开销；
- 如果循环模块足够小，可以将其完全放入高速缓存（如 LPU/Groq 芯片的 SRAM），实现极致低延迟。

7.4.3 不同应用场景的最佳架构差异

- **智能体/多轮对话**：KV 缓存效率至关重要。DeepSeek 的 MLA（Multi-head Latent Attention）将 KV 缓存压缩得相当彻底；也可以使用 FP8 或 FP4 格式处理 KV 缓存。这些因素对智能体工作流的影响尤为显著。另一个关键选择是因果注意力 vs 非因果注意力。

- **大规模批处理：**因果注意力的重要性下降。谷歌在搜索中可能仍在使用 BERT 模型——因为只需要进行一次大规模的双向注意力处理，提取向量后存入数据库即可随意使用。T5 等编码器-解码器模型也曾被用于双向处理和生成。

7.5 结语

Dan Fu 的讲座向我们展示了一个正在快速演进的领域。推理系统的发展仍处于早期阶段——正如他所说，“10 年后回顾今天，人们可能会说：他们为什么还在讨论这些？这不是很显然吗？”

但对于身处当下的我们而言，这正是一个充满机会的时代。无论是工程优化、内核编程，还是模型架构的根本性重新思考，每一个层面都有大量开放问题和创新空间等待探索。

“如果你理解推理本身以及推理引擎，以及理解支撑核心技术的 GPU 内核，你就能在机器学习算法的全栈层面实现创新。”

——Dan Fu